

QUESTION 1

1. Grid and Initialisation

A uniform 51×51 Cartesian grid is set up over the unit square. All fields (ψ , ω , u , v) are initialised to zero, corresponding to fluid at rest:

```
L = 1.0; U = 1.0; Re = 100
Nx = 51; Ny = 51
dx = L / (Nx - 1)      # dx = 0.02
dy = L / (Ny - 1)      # dy = 0.02
x = np.linspace(0, L, Nx)
y = np.linspace(0, L, Ny)
X, Y = np.meshgrid(x, y)

psi = np.zeros((Ny, Nx))
omega = np.zeros((Ny, Nx))
u = np.zeros((Ny, Nx))
v = np.zeros((Ny, Nx))
```

1.1 Velocity Recovery and Lid Boundary Condition

At the start of each outer iteration, the velocity field is updated from the current stream function using central differences. The top-lid velocity is then hard-set:

```
# Interior velocities from stream function
u[1:-1, 1:-1] = (psi[2:, 1:-1] - psi[:-2, 1:-1]) / (2 * dy) # u = dψ/dy
v[1:-1, 1:-1] = -(psi[1:-1, 2:] - psi[1:-1, :-2]) / (2 * dx) # v = -dψ/dx

# Moving top lid (Dirichlet)
u[-1, :] = U      # = 1.0
v[-1, :] = 0
```

1.2 Adaptive Time-Step Calculation

To maintain numerical stability, the time step dt is computed dynamically each iteration as the minimum of the convective and diffusive CFL limits, scaled by a safety factor of 0.5:

```
umax = np.max(np.abs(u)) + 1e-8
vmax = np.max(np.abs(v)) + 1e-8

dt_conv = min(dx / umax, dy / vmax) # convective CFL
dt_diff = Re * min(dx**2, dy**2) / 4 # diffusive stability limit

dt = 0.5 * min(dt_conv, dt_diff) # conservative combined limit
```

The diffusive limit scales as $Re \cdot \Delta x^2/4$, which for $Re = 100$ and $\Delta x = 0.02$ gives $dt_diff \approx 0.01$. The safety factor of 0.5 ensures both conditions are comfortably satisfied throughout the transient.

1.3 Vorticity Transport – Explicit Upwind Scheme

The vorticity convection terms are discretised using first-order upwind (donor-cell) differencing: the gradient is taken from upstream of the local velocity direction. This provides unconditional stability for the convective term but introduces first-order numerical diffusion:

```
for i in range(1, Ny - 1):
    for j in range(1, Nx - 1):

        # Upwind x-derivative of vorticity
        if u[i, j] > 0:
            domega_dx = (omega_old[i, j] - omega_old[i, j-1]) / dx
        else:
            domega_dx = (omega_old[i, j+1] - omega_old[i, j]) / dx

        # Upwind y-derivative of vorticity
        if v[i, j] > 0:
            domega_dy = (omega_old[i, j] - omega_old[i-1, j]) / dy
        else:
            domega_dy = (omega_old[i+1, j] - omega_old[i, j]) / dy

        conv = u[i, j]*domega_dx + v[i, j]*domega_dy

        # Central difference Laplacian for diffusion
        lap_omega = ((omega_old[i+1, j] - 2*omega_old[i, j] + omega_old[i-1, j]) /
dy**2
                    + (omega_old[i, j+1] - 2*omega_old[i, j] + omega_old[i, j-1]) /
dx**2)
        diff = lap_omega / Re

        # Explicit forward-Euler update
        omega[i, j] = omega_old[i, j] + dt * (diff - conv)
```

1.4 Vorticity Boundary Conditions (Thom's Formula)

Wall vorticity is set using Thom's second-order formula, which relates the wall vorticity to the stream function in the first interior row/column. The moving lid introduces an additional term proportional to U :

```
# Top wall (moving lid): extra -2U/dy term from lid velocity
omega[-1, :] = -2 * psi[-2, :] / dy**2 - 2 * U / dy

# Bottom wall (stationary)
omega[0, :] = -2 * psi[1, :] / dy**2

# Left wall (stationary)
omega[:, 0] = -2 * psi[:, 1] / dx**2

# Right wall (stationary)
omega[:, -1] = -2 * psi[:, -2] / dx**2
```

The term $-2U/dy$ in the top-wall condition arises because the lid moves at speed U ; for a stationary wall, only the stream-function term is present. This ensures that the no-slip condition at the moving lid is correctly incorporated into the vorticity field.

1.5 Stream Function Poisson Solve (Gauss-Seidel)

At each outer iteration, the Poisson equation $\nabla^2\psi = -\omega$ is solved by up to 200 Gauss-Seidel sweeps. The stream function is zero on all four walls (no-penetration, closed cavity):

```
for k in range(200):
    psi_old = psi.copy()
    for i in range(1, Ny - 1):
        for j in range(1, Nx - 1):
            psi[i, j] = ( (psi[i+1, j] + psi[i-1, j]) / dy**2
                          + (psi[i, j+1] + psi[i, j-1]) / dx**2
                          + omega[i, j] )
                        / (2/dx**2 + 2/dy**2)

    # Boundary conditions: all walls, psi = 0
    psi[0, :] = 0; psi[-1, :] = 0
    psi[:, 0] = 0; psi[:, -1] = 0

    # Early exit if Poisson converged
    if np.max(np.abs(psi - psi_old)) < tol:
        break
```

The inner Gauss-Seidel loop exits early when the stream function change falls below 10^{-6} , avoiding unnecessary work when the Poisson equation is already well converged.

1.6 Global Convergence Monitoring

The outer loop checks convergence by comparing the vorticity field before and after each iteration. The residual is printed every 100 steps:

```
res = np.max(np.abs(omega - omega_old_global))
if iteration % 100 == 0:
    print(f'Iteration = {iteration}, Residual = {res:.3e}')
if res < tol: # tol = 1e-6
    print('Steady state reached')
    break
```

2. Results and Discussion

2.1 Stream Function Contours

The stream function contour plot visualises the flow topology through iso-lines of ψ . Fluid particles travel along these lines, so the pattern directly reveals the structure of the circulation inside the cavity:

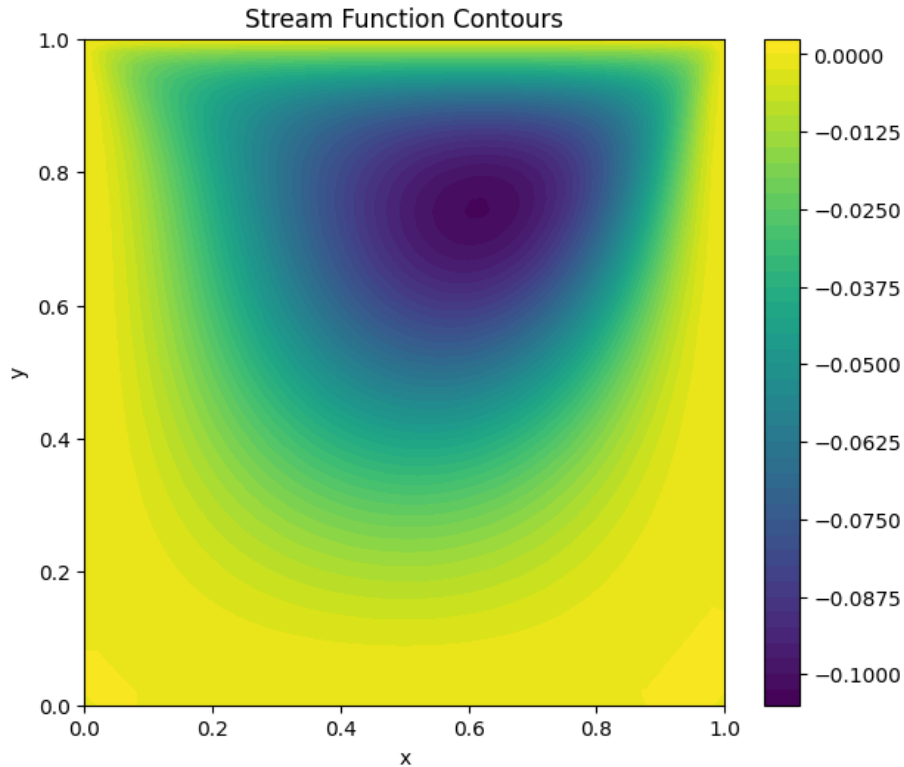


Figure 1: Stream function contours (ψ) for lid-driven cavity flow at $Re = 100$.

Primary vortex: A single large, roughly circular recirculating vortex occupies most of the cavity interior. The tightly packed contour lines near the top and side walls indicate high velocity gradients in those regions. The vortex centre is located slightly above and to the right of the geometric centre, consistent with published results at $Re = 100$ (Ghia et al., 1982).

Contour spacing and flow speed: Regions where contour lines are closely spaced correspond to high stream function gradients and therefore high flow speeds (since $u = \partial\psi/\partial y$, $v = -\partial\psi/\partial x$). The densest packing occurs just beneath the moving lid, driven directly by the lid shear.

Corner regions: The contours near the bottom corners are widely spaced, indicating slow-moving fluid and the presence of small secondary vortices that are not resolved as distinct features at $Re = 100$ but become visible at higher Re .

2.2 Velocity Vector Field

The quiver plot shows the direction and relative magnitude of the velocity vector (u, v) at every grid point:

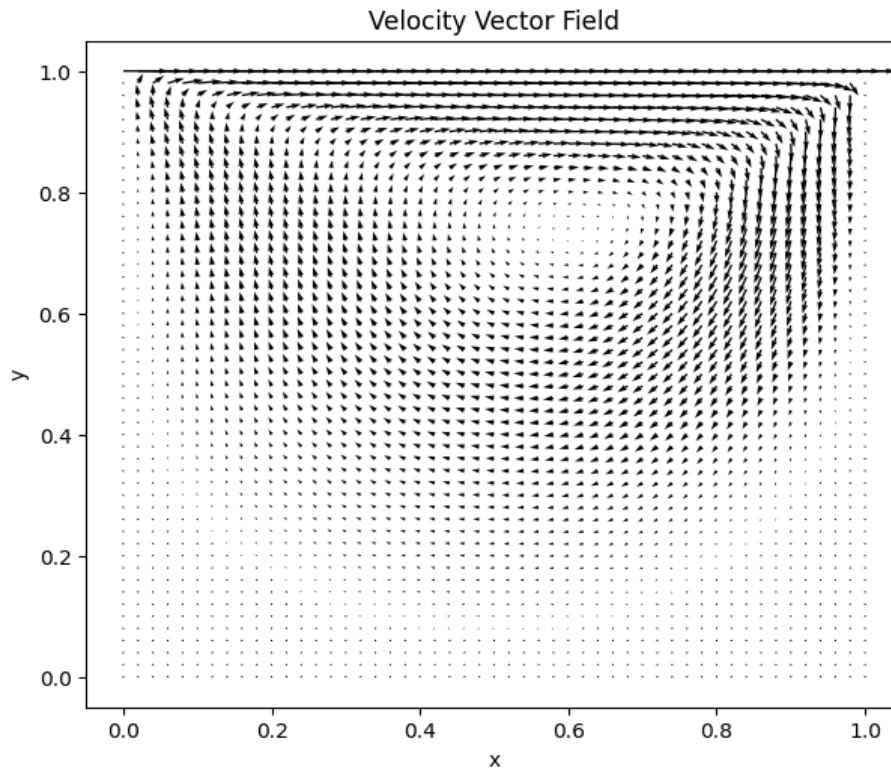


Figure 2: Velocity vector field for lid-driven cavity flow at $Re = 100$.

Lid-driven shear layer: The long horizontal arrows at the top of the domain confirm the uniform lid velocity $U = 1$ m/s directed to the right. Below the lid, the velocity drops sharply, defining a thin shear layer that drives the cavity circulation.

Recirculation pattern: Moving clockwise (when viewed with y upward): the lid drags fluid rightward at the top; fluid descends along the right wall; flows leftward near the bottom; and rises along the left wall. The vector magnitudes decrease progressively away from the lid, confirming that momentum is communicated into the interior by viscous diffusion.

Near-wall behaviour: All velocity vectors along the stationary walls (bottom, left, right) are essentially zero, verifying that the no-slip boundary conditions are correctly enforced.

Vortex core: Near the centre of the cavity the vectors are small and change direction gradually, indicating the low-speed core region of the primary vortex. The centre is shifted slightly toward the upper-right quadrant as expected for $Re = 100$.

QUESTION 2

1. Boundary Conditions

Boundary	Condition Applied
Inlet ($x = 0$)	$u = U, v = 0; \quad \psi = U \cdot y$
Bottom wall ($y = 0$)	$u = v = 0; \quad \psi = 0; \quad \omega = -2(\psi_1 - \psi_0)/\Delta y^2$ (Thom's formula)
Top wall ($y = Ly$)	$u = v = 0; \quad \psi = U \cdot Ly; \quad \omega = -2(\psi_{\{Ny-2\}} - \psi_{\{Ny-1\}})/\Delta y^2$
Outlet ($x = Lx$)	Zero-gradient (Neumann): $\psi_N = \psi_{\{N-1\}}; \quad \omega_N = \omega_{\{N-1\}}$

2. Numerical Method

2.1 Spatial Discretisation

The domain is covered by a uniform Cartesian grid with N_x nodes in the x -direction (spacing $\Delta x = Lx/(N_x-1)$) and N_y nodes in the y -direction (spacing $\Delta y = Ly/(N_y-1)$). All spatial derivatives are approximated by second-order central differences.

The Poisson equation for ψ is updated at each interior node by a Gauss-Seidel point iteration:

Poisson update (Gauss-Seidel):

$$\psi(i,j) = 0.25 \times [\psi(i+1,j) + \psi(i-1,j) + \psi(i,j+1) + \psi(i,j-1) + \omega(i,j) \cdot \Delta x \cdot \Delta y]$$

The vorticity transport equation is solved by an explicit pseudo-time relaxation:

Vorticity relaxation update:

$$\omega(i,j) \leftarrow \omega(i,j) + \alpha [- (u \partial \omega / \partial x + v \partial \omega / \partial y) + \nu \nabla^2 \omega]$$

The relaxation parameter $\alpha = 0.01$ is chosen to ensure stability for the full range of Re tested.

2.2 Solution Algorithm

1. Initialise $\psi = U \cdot y$ and $\omega = 0$ (uniform plug-flow initial condition).
2. Sweep the interior with the Gauss-Seidel update to obtain a new ψ field.
3. Apply inlet, wall, and outlet boundary conditions on ψ .
4. Compute u and v from the updated ψ via central differences (Eq. 3).
5. Enforce velocity boundary conditions (no-slip at walls, uniform at inlet, Neumann at outlet).
6. Update wall vorticity using Thom's formula; apply outlet Neumann condition on ω .
7. Advance the interior vorticity field with the relaxation update (Eq. 2 discrete).
8. Clamp ω to $[-10^6, +10^6]$ to prevent overflow.
9. Check convergence: $\max|\Delta u| < 10^{-4}$. If met, stop; otherwise return to step 2.

2.3 Entry Length Detection

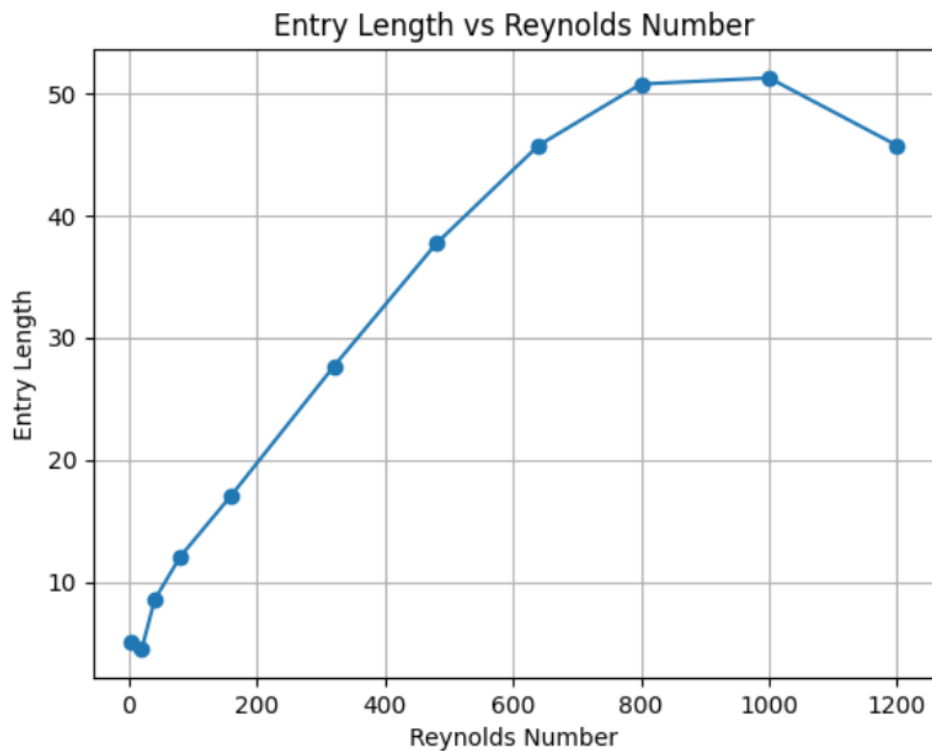
Once the velocity field has converged, the entry length L_e is identified as the first streamwise grid location x_i where the maximum streamwise velocity gradient across the channel satisfies the fully-developed criterion:

Fully-developed criterion:

$\max_j |\partial u / \partial x|(i, j) < 10^{-3}$ If the criterion is never met, L_e defaults to L_x (channel length).

The hydraulic diameter for a parallel-plate channel is $D_h = 2L_y$ (twice the gap), and the Reynolds number is $Re = U \cdot D_h / \nu$.

```
Enter channel length: 100
Enter channel height: 2
Enter number of grid points in x direction: 200
Enter number of grid points in y direction: 20
Enter kinematic viscosity: 0.1
```



Part 2

1.1 Grid Setup and Inlet Velocity Derivation

After reading user inputs, the code computes the uniform Cartesian grid spacings and derives the inlet velocity for each case from the definition of the Reynolds number:

```
Dh = 2 * Ly                                # hydraulic diameter for parallel plates
U1 = (Re1 * nu) / Dh                       # inlet velocity for Re1
U2 = (Re2 * nu) / Dh                       # inlet velocity for Re2

dx = Lx / (Nx - 1)
dy = Ly / (Ny - 1)
x = np.linspace(0, Lx, Nx)
y = np.linspace(0, Ly, Ny)
```

For the values used here: $U1 = 5 \times 0.1 / 4 = 0.125$ m/s and $U2 = 25 \times 0.1 / 4 = 0.625$ m/s.

1.2 Initialisation – Plug-Flow Starting Condition

The solver begins from a physically motivated initial guess: uniform plug flow throughout the channel. Both the axial velocity and the stream function are set to their inlet values at every node:

```
for j in range(Ny):
    for i in range(Nx):
        u_velocity[j][i] = U           # uniform inlet speed everywhere
        psi[j][i] = U * y[j]          # psi satisfying du/dy = U
```

Starting from the plug-flow state reduces the number of iterations needed for convergence and avoids divergence that can occur with a zero-velocity initial field.

1.3 Stream Function – Poisson Equation Solve

The stream function is governed by the Poisson equation $\nabla^2 \psi = -\omega$. Its discrete form is solved by a Gauss-Seidel point-iteration, updating each interior node with the average of its four neighbours plus a source term from the local vorticity:

```
for j in range(1, Ny - 1):
    for i in range(1, Nx - 1):
        east = psi[j][i + 1]
        west = psi[j][i - 1]
        north = psi[j + 1][i]
        south = psi[j - 1][i]
        source = omega[j][i] * dx * dy
        psi[j][i] = 0.25 * (east + west + north + south + source)
```

Boundary conditions are applied after every sweep: the inlet column is fixed to the plug-flow distribution ($\psi = U \cdot y$); both walls hold constant values ($\psi = 0$ at $y = 0$, $\psi = U \cdot Ly$ at $y = Ly$);

and the outlet column is set equal to its upstream neighbour (Neumann / zero-gradient condition).

1.4 Velocity Recovery from the Stream Function

Once ψ is updated, the velocity components follow from the kinematic relations $u = \partial\psi/\partial y$ and $v = -\partial\psi/\partial x$, discretised with central differences:

```
u_velocity[j][i] = (psi[j+1][i] - psi[j-1][i]) / (2 * dy)    # u = dψ/dy
v_velocity[j][i] = -(psi[j][i+1] - psi[j][i-1]) / (2 * dx)  # v = -dψ/dx
```

No-slip conditions ($u = v = 0$) are enforced at both walls; the inlet column is reset to the uniform speed U ; and zero-gradient conditions are applied at the outlet.

1.5 Vorticity Transport and Wall Vorticity (Thom's Formula)

Wall vorticity is computed using Thom's formula, which relates the vorticity at the wall to the stream function values in the first interior row:

```
# Bottom wall
omega[0][i] = -2 * (psi[1][i] - psi[0][i]) / (dy * dy)

# Top wall
omega[Ny-1][i] = -2 * (psi[Ny-2][i] - psi[Ny-1][i]) / (dy * dy)

# Outlet: zero-gradient
omega[j][Nx-1] = omega[j][Nx-2]
```

Interior vorticity is advanced using an explicit relaxation update with backward differences for the convective terms:

```
dwdx = (omega[j][i] - omega[j][i-1]) / dx    # upwind (backward) difference
dwdy = (omega[j][i] - omega[j-1][i]) / dy
convection = u_velocity[j][i]*dwdx + v_velocity[j][i]*dwdy

diffusion_x = (omega[j][i+1] - 2*omega[j][i] + omega[j][i-1]) / dx**2
diffusion_y = (omega[j+1][i] - 2*omega[j][i] + omega[j-1][i]) / dy**2
diffusion = nu * (diffusion_x + diffusion_y)

omega_new = omega[j][i] + alpha * (-convection + diffusion)    # alpha = 0.1
omega[j][i] = 0.9 * omega[j][i] + 0.1 * omega_new            # strong blending
```

The 90-10 blending (old-to-new) acts as heavy under-relaxation, keeping the vorticity update stable across the full Re range at the cost of slower convergence.

1.6 Convergence Check

After each iteration the maximum absolute change in axial velocity is evaluated. The loop exits when this falls below the tolerance of 10^{-4} :

```
error = np.max(np.abs(u_velocity - u_old))
if error < tolerance:      # tolerance = 1e-4
    print('Converged for U =', U)
    break
```

Both cases converged successfully:

Converged for U = 0.125 (Re1 = 5) Converged for U = 0.625 (Re2 = 25)

1.7 Profile Extraction and Plot

Once the velocity field has converged, the profile at the requested x-station is extracted by identifying the nearest grid column index and reading the u-velocity across all y-nodes:

```
i_profile = int(x_location / dx)      # nearest column to x* = 3

profile1 = [u1[j][i_profile] for j in range(Ny)]
profile2 = [u2[j][i_profile] for j in range(Ny)]

plt.plot(profile1, y, label=f'Re = {Re1}')
plt.plot(profile2, y, label=f'Re = {Re2}')
plt.xlabel('u velocity')
plt.ylabel('y')
plt.title('Velocity Profiles at Different Reynolds Numbers')
```

2. Results and Discussion

2.1 Obtained Plot

The figure below shows the streamwise velocity profiles $u(y)$ extracted at $x^* = 3$ for $Re = 5$ (blue) and $Re = 25$ (orange):

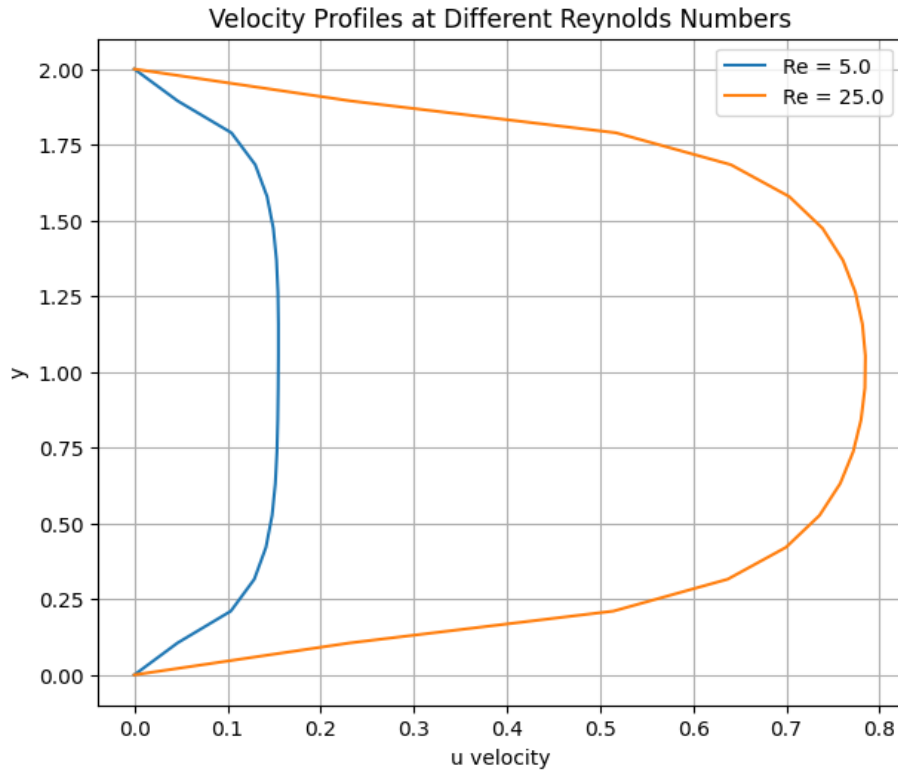


Figure 1: u -velocity profiles across channel height at $x^* = 3$, for $Re = 5$ and $Re = 25$.

2.2 Profile Shape and Development

Both profiles lie within the hydrodynamic entry region at $x^* = 3$. **Neither has yet reached the fully-developed parabolic (Poiseuille) shape** $u(y) = 6\bar{U}y(Ly-y)/Ly^2$, because the entry length for both Reynolds numbers far exceeds $x^* = 3$.

At this station both curves retain a broad, flattened central core inherited from the uniform inlet condition, with the velocity dropping sharply to zero only in the immediate vicinity of the walls.

2.3 Effect of Reynolds Number on Profile Development

Low Re (Re = 5, U = 0.125 m/s): At low Re, viscous diffusion dominates over inertia. Momentum is transported rapidly across the channel, causing the boundary layers near both walls to grow quickly. As a result the Re = 5 profile is noticeably more rounded at $x^* = 3$ — the flat core is narrower and the velocity has already begun to redistribute toward the parabolic shape.

Higher Re (Re = 25, U = 0.625 m/s): At Re = 25, inertia resists lateral momentum diffusion. The wall boundary layers grow more slowly, so the central core remains broad and nearly uniform at $x^* = 3$. The profile looks closer to plug flow than the Re = 5 case at the same axial station.

2.4 Velocity Magnitudes and Dimensional Plot

Because the profiles are plotted in dimensional units (m/s rather than u/U), the two curves occupy different horizontal ranges: the Re = 25 curve extends to roughly five times the peak value of the Re = 5 curve, simply because $U_2 = 5U_1$. In a normalised plot (u/U vs. y/Ly) the

profiles would be directly comparable and the $Re = 25$ curve would appear flatter, confirming slower development.

2.5 Symmetry and Boundary Condition Verification

- Both profiles are symmetric about the channel centreline $y = 1$, as required by the symmetric no-slip walls, confirming that no artificial asymmetry has been introduced by the numerical scheme.
- Both profiles correctly return $u = 0$ at $y = 0$ and $y = 2$ (top and bottom walls), verifying correct enforcement of the no-slip condition.

The outlet Neumann condition (zero streamwise gradient) does not visibly contaminate the interior solution at $x^* = 3$, which is well upstream of the outlet at $x = 100$.

QUESTION 3

1. Marching in x direction

```
for i in range(0, Nx - 1):
    for j in range(1, Ny - 1):
        du_dy = (u_velocity[j+1][i] - u_velocity[j-1][i]) / (2 * dy)
        d2u_dy2 = (u_velocity[j+1][i] - 2*u_velocity[j][i] +
u_velocity[j-1][i]) / (dy * dy)

        if u_velocity[j][i] < 1e-6:
            u_local = 1e-6
        else:
            u_local = u_velocity[j][i]

        du_dx = (nu * d2u_dy2 - v_velocity[j][i] * du_dy) / u_local

        u_velocity[j][i+1] = u_velocity[j][i] + dx * du_dx
    ...
```

- ``du_dy`` and ``d2u_dy2`` are computed with **central differences** in y .
- ``du_dx`` is obtained by rearranging the momentum equation.
- The velocity at the next x -station is computed by **forward Euler integration** in x .
- A guard ``u_local = 1e-6`` prevents division by zero near the wall.

2. Key Code: Continuity for v-Velocity

```
```python
Continuity equation
for j in range(1, Ny):
 du_dx_local = (u_velocity[j][i+1] - u_velocity[j][i]) / dx
 ...
 v_velocity[j][i+1] = v_velocity[j-1][i+1] - dy * du_dx_local
 ...
```

The **continuity equation** ( $\partial u / \partial x + \partial v / \partial y = 0$ ) is integrated upward from the wall ( $v = 0$  at wall) to obtain the vertical velocity component  $v$  at each new  $x$ -station.

### 3. Key Code: Boundary Layer Thickness Detection

```
```python
for i in range(Nx):
    delta = 0
    for j in range(Ny):
        velocity_ratio = u_velocity[j][i] / U
        if velocity_ratio >= 0.99:
            delta = y[j]
            break
    delta_list.append(delta)
```
```

The **boundary layer thickness  $\delta$**  is defined as the  $y$ -location where the local velocity first reaches 99% of the free-stream velocity ( $u/U = 0.99$ ). This is the standard engineering definition.

